

Believe设计思路

靶机名称: Believe

作者: Gropers

靶机ID: 661

难度: baby

靶机地址: <https://maze-sec.com>

靶机IP: 192.168.1.150

攻击机IP: 192.168.1.195(Kali Linux)

信息收集与外网突破

端口扫描:

```
(root@Gropers)-[~]
└─# nmap -A -sT -p- 192.168.1.150
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-23 04:31 -0400
Nmap scan report for 192.168.1.150
Host is up (0.0011s latency).
Not shown: 65531 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.67 ((Unix))
|_http-title: Login
|_http-server-header: Apache/2.4.67 (Unix)
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_     httponly flag not set
2222/tcp  open  ssh      OpenSSH 10.3 (protocol 2.0)
54321/tcp open  ssh      OpenSSH 10.3 (protocol 2.0)
MAC Address: 08:00:27:AC:0E:5E (Oracle VirtualBox virtual NIC)
No exact OS matches for host (If you know what OS is running on it, see
https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=5/23%OT=22%CT=1%CU=38292%PV=Y%DS=1%DC=D%G=Y%M=080027%T
OS:M=6A1165E6%P=x86_64-pc-linux-gnu)SEQ(SP=100%GCD=1%ISR=104%TI=Z%CI=Z%II=I
OS:%TS=22)SEQ(SP=104%GCD=1%ISR=10C%TI=Z%CI=Z%II=I%TS=21)SEQ(SP=106%GCD=1%IS
OS:R=107%TI=Z%CI=Z%TS=21)SEQ(SP=106%GCD=1%ISR=10B%TI=Z%CI=Z%II=I%TS=21)SEQ(
OS:SP=F9%GCD=1%ISR=10F%TI=Z%CI=Z%II=I%TS=21)OPS(O1=M5B4ST11NW9%O2=M5B4ST11N
OS:W9%O3=M5B4NNT11NW9%O4=M5B4ST11NW9%O5=M5B4ST11NW9%O6=M5B4ST11)WIN(W1=FE88
OS:%W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M5B4
OS:NNSNW9%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R
OS:=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=
OS:AR%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=
OS:40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID
OS:=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

Network Distance: 1 hop

TRACEROUTE

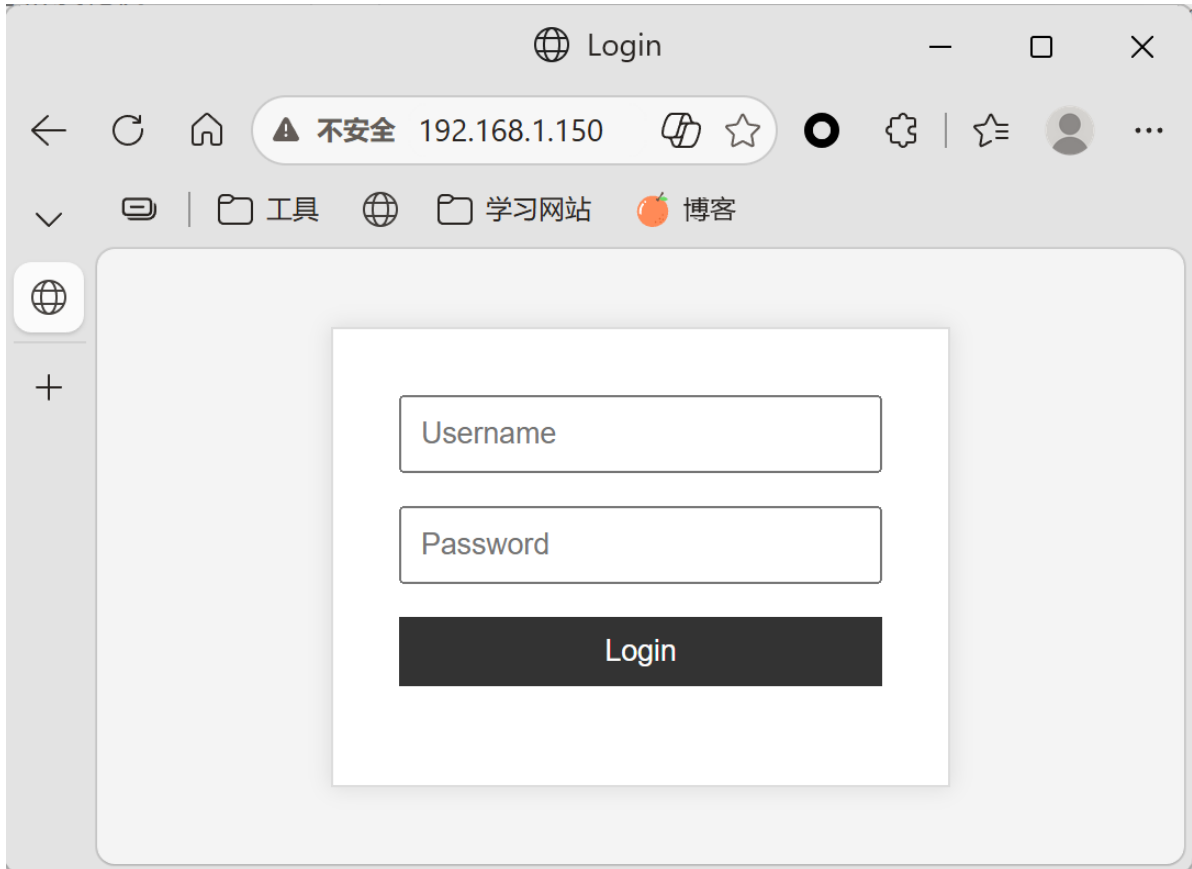
HOP	RTT	ADDRESS
-----	-----	---------

```
1 1.06 ms 192.168.1.150
```

OS and Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .
Nmap done: 1 IP address (1 host up) scanned in 24.77 seconds

靶机一共开启了四个端口

80 端口用于web页面，进入之后是一个登录页面



gobuster扫一下目录

```
└─(root@kali)-[~]
└─# gobuster dir -u http://192.168.1.150 -w
    /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,txt,html,bak
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url:                http://192.168.1.150
[+] Method:             GET
[+] Threads:            10
[+] Wordlist:            /usr/share/wordlists/dirbuster/directory-list-2.3-
medium.txt
[+] Negative status codes: 404
[+] User Agent:          gobuster/3.8.2
[+] Extensions:         txt,html,bak,php
[+] Timeout:            10s
=====
Starting gobuster in directory enumeration mode
=====
index.php                (Status: 200) [Size: 772]
```

```
dashboard.php      (Status: 302) [Size: 0] [--> index.php]
yu.txt             (Status: 200) [Size: 27]
server-status      (Status: 403) [Size: 318]
Progress: 1102790 / 1102790 (100.00%)

=====
Finished
=====
```

在这可以发现除了登录页面还有一个 `dashboard` 控制面板

在这里出现了一个设计失误，导致yu.txt直接被扫描器扫出来了

结果导致选手没有通过登陆页面直接获取到了准备的提示信息

22、2222、54321 开放了三个ssh端口

其中 22 端口下有一组banner凭证

```
(root@Gropers)-[~]
# ssh root@192.168.1.150
=====
                        admin
                    yubao9694482664
=====
```

通常来说选手得到这组凭证之后一定回去网页的登录口进行尝试

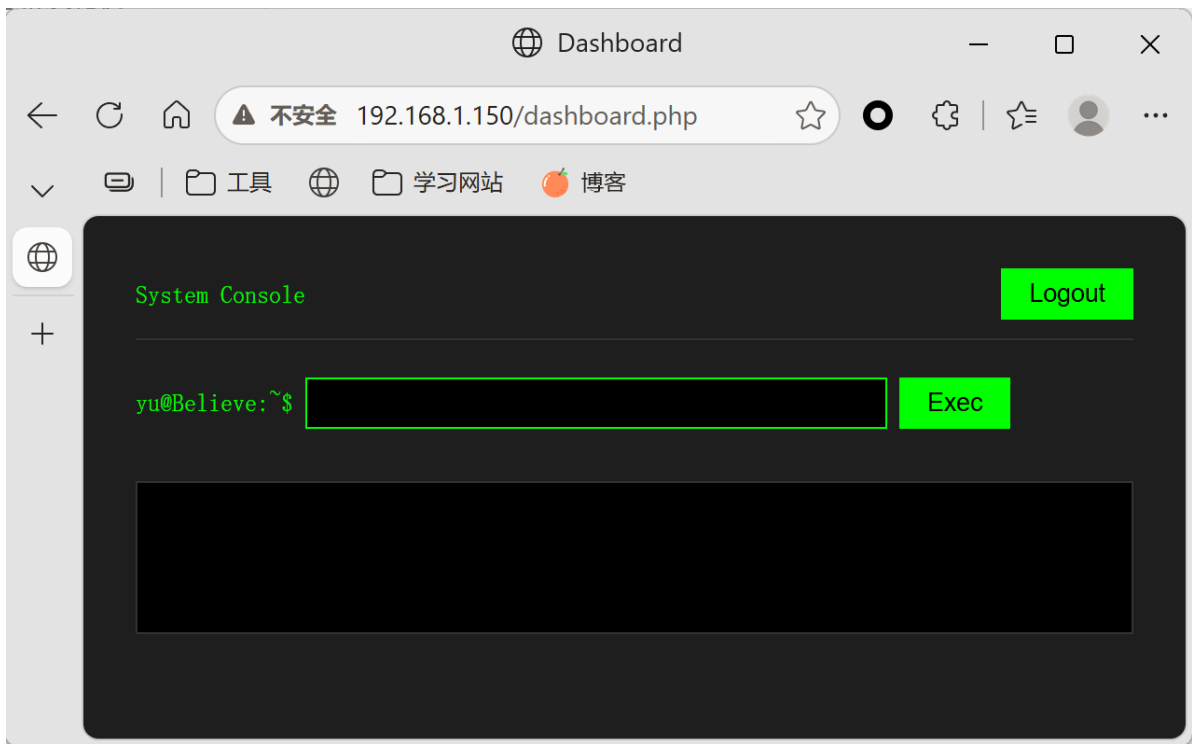
但是输入这组凭证却无法进入

这其实就是这个靶机的制作初衷

就像是ssh登录一样哪怕你输入错误，也会给你输入三次的机会

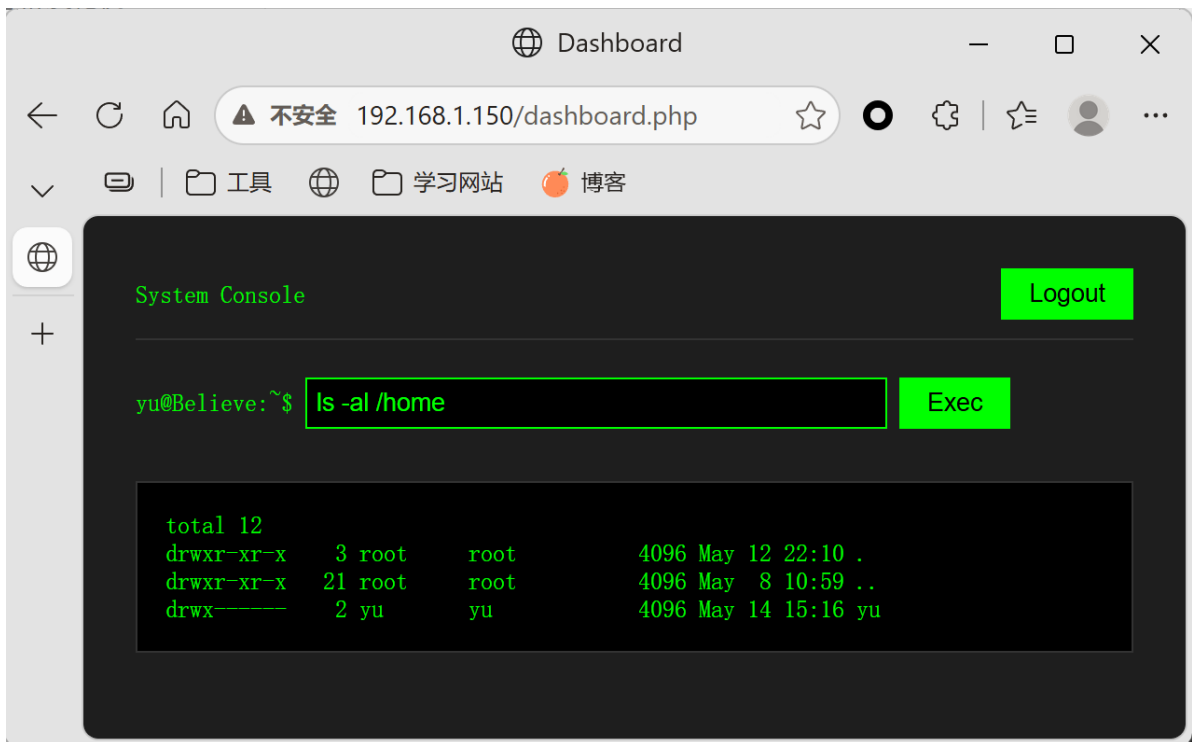
需要连续 3 次使用 admin:yubao9694482664 登录（同一次会话）

就可以进入到 `dashboard` 控制面板

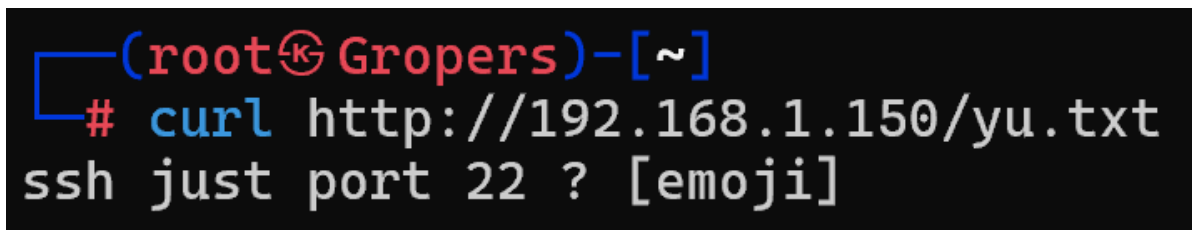


首先一进来就可以得到存在一 `yu` 用户，我直接将它放在了网页上

当然也可以直接通过 `ls -al /home` 得到存在的这个用户



网页下给出的提示文件 `yu.txt` 也同样给出了提示



运行 `./be`，这同样是一个口令验证程序

但是可以将它拉到本地进行一个逆向反编译

反汇编分析

main 函数 (0x401b1a)

- 1. 在栈上设置 20 字节的硬编码密钥目标
- 2. 调用 `generate_key()` 生成 7 字节 RC4 密钥
- 3. 打印 "请输入系统密钥："
- 4. 使用 `scanf("%255s")` 读取用户输入
- 5. 计算输入长度 `strlen`
- 6. 调用 `rc4_init(key, 7)` 初始化 RC4
- 7. 调用 `rc4_crypt(state, input, length)` 解密输入
- 8. 检查长度是否为 20 (0x14)
- 9. 逐字节比较解密后的输入与硬编码的 20 字节目标
- 10. 匹配则调用 `get_shell()` 并打印 "验证成功。"
- 11. 不匹配则打印 "权限不足。"

generate_key 函数 (0x401a24)

```
movl    $0xc3c6cfc8,-0xb(%rbp)    # 栈上存储加密数据
movl    $0xcfdccfc3,-0x8(%rbp)
# 循环 7 次 (i = 0..6), 每字节 XOR 0xaa
```

推导过程：

偏移	原始字节	XOR 0xaa	ASCII
0	0xc8	0x62	b
1	0xcf	0x65	e
2	0xc6	0x6c	l
3	0xc3	0x69	i
4	0xcf	0x65	e
5	0xdc	0x76	v
6	0xcf	0x65	e

RC4 密钥 = "believe"

get_shell 函数 (0x401a77)

```
# 解码 "/bin/bash" (bitwise NOT 每字节)
movabs $0x8c9e9dd091969dd0,%rax    # NOT → 0x62696e2f = "/bin"
movw   $0xff97,-0x6(%rbp)          # NOT → 0x0068 = "h\0"
# → 解码结果: "/bin/bash"

setuid(0)                          # 设置 UID 为 root
setgid(0)                          # 设置 GID 为 root
puts("[+] 正在请求 Believe 核心权限...")
execve("/bin/bash", {"/bin/bash", "-p", NULL}, NULL)
```

计算密码get_shell

RC4 架构:

- 密钥: "believe" (7 字节)
- 对 20 字节硬编码目标进行 RC4 加密 (因为程序会解密用户输入后比对)

```
target = bytes([
    0x24, 0x53, 0x27, 0x50, 0x78, 0x4b, 0x9d, 0xab,
    0xab, 0x7e, 0xf0, 0xa1, 0x49, 0xf2, 0xea, 0x7e,
    0xec, 0x6c, 0x82, 0x77
])
rc4_key = b"believe"

# RC4 加密目标得到密码
password = rc4_crypt(rc4_init(rc4_key), target)
# → b"Believe_in_yourself!" (20 字节)
```

最后得到了口令就是Believe_in_yourself!

输入口令就会执行一个 /bin/bash

```
yu@Believe:~$ ./be
请输入系统密钥: Believe_in_yourself!
验证成功。
[+] 正在请求 Believe 核心权限...
Believe:/home/yu# whoami
root
Believe:/home/yu# id
uid=0(root) gid=0(root) groups=1000(yu)
Believe:/home/yu# cat /root/root.txt
flag{Believe in yourself forever!!!}
Believe:/home/yu#
```

得到最终的root flag

Web分析

回过头来看 `/var/www/html` 文件夹下的文件，分析 `index` 和 `dashboard`

index.php

```
<?php
error_reporting(0);
session_start();
$m = '';
if (isset($_SESSION['auth']) && $_SESSION['auth'] === true) {
    header('Location: dashboard.php');
    exit;
}
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $u = $_POST['u'] ?? '';
    $p = $_POST['p'] ?? '';
    if ($u === 'admin' && $p === 'yubao9694482664') {
        $_SESSION['c'] = ($_SESSION['c'] ?? 0) + 1;
        if ($_SESSION['c'] >= 3) {
            $_SESSION['auth'] = true;
            header('Location: dashboard.php');
            exit;
        }
    } else {
        $_SESSION['c'] = 0;
    }
    $m = '';
}
?>
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Login</title>
<style>
body { display: flex; justify-content: center; align-items: center; height:
100vh; margin: 0; background: #f4f4f4; font-family: sans-serif; }
form { background: #fff; padding: 30px; border: 1px solid #ddd; box-shadow: 0 0
10px rgba(0,0,0,0.1); }
input { margin-bottom: 15px; padding: 8px; width: 200px; display: block; }
input[type="submit"] { width: 100%; cursor: pointer; background: #333; color:
#fff; border: none; }
.e { color: red; font-size: 14px; text-align: center; }
</style>
</head>
<body>
<form method="POST">
<input type="text" name="u" placeholder="Username">
<input type="password" name="p" placeholder="Password">
<input type="submit" value="Login">
<?php if ($m !== '') { echo "<div class='e'>" . htmlspecialchars($m) . "</div>";
} ?>
</form>
</body>
```

```
</html>
```

需要连续 3 次使用 `admin:yubao9694482664` 登录（同一次会话），才能获得 `auth=true` 并访问 `dashboard.php`。

dashboard.php

```
<?php
error_reporting(0);
session_start();
if (isset($_POST['logout'])) {
    session_destroy();
    header('Location: index.php');
    exit;
}
if (!isset($_SESSION['auth']) || $_SESSION['auth'] !== true) {
    header('Location: index.php');
    exit;
}
$r = '';
if (isset($_POST['cmd'])) {
    $c = trim($_POST['cmd']);
    if (preg_match('/^ls(\s+.*?)?$/ ', $c) || preg_match('/^cat\s+.*\.txt$/ ', $c))
    {
        if (!preg_match('/[;<`\\n]/ ', $c)) {
            $r = shell_exec($c);
        }
    }
}
?>
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Dashboard</title>
<style>
body { background: #1e1e1e; color: #00ff00; font-family: monospace; padding:
20px; }
.top-bar { display: flex; justify-content: space-between; align-items: center;
margin-bottom: 20px; border-bottom: 1px solid #333; padding-bottom: 10px; }
input[type="text"] { background: #000; color: #00ff00; border: 1px solid #00ff00;
padding: 5px; width: 300px; }
input[type="submit"] { background: #00ff00; color: #000; border: none; padding:
6px 15px; cursor: pointer; }
.logout-btn { background: #ff0000; color: #fff; border: none; padding: 6px 15px;
cursor: pointer; }
pre { background: #000; padding: 15px; border: 1px solid #333; min-height: 50px;
}
</style>
</head>
<body>
<div class="top-bar">
    <span>System Console</span>
    <form method="POST" style="margin: 0;">
        <input type="submit" name="logout" value="Logout" class="logout-btn">
```

```
</form>
</div>
<form method="POST">
<span class="prompt">yu@Believe:~$</span>
<input type="text" name="cmd" autofocus autocomplete="off">
<input type="submit" value="Exec">
</form>
<br>
<pre><?php echo htmlspecialchars($r, ENT_QUOTES, 'UTF-8'); ?></pre>
</body>
</html>
```

认证后可以执行 `ls` 和 `cat *.txt` 命令，但存在过滤（禁止 `;&$><` 等字符）。

倒是可以用多文件规避正则实现读任意文件

```
if (preg_match('/^ls\s+.*$/', $c) || preg_match('/^cat\s+.*\.txt$/', $c))
{
    if (!preg_match('/[;&$><`\\n]/', $c)) {
        $r = shell_exec($c);
    }
}
```

总结

这个靶机哪怕经过了多轮测试，还是存在一些问题

现在ai的不断强大，对于静态二进制文件还是冲击较大，选手可以通过ai快速完成最后对于二进制文件的反编译快速得到口令，只有动态调试的文件暂时还算良好，但是相信不久以后ai也能解决类似的问题

这个靶机制作的一个初衷就是之前听到的一个玩笑话

当你输入正确密码后也不能进入页面才能正在防护爆破，因为真正知道密码的人一定还会输入几遍 (emoji) [嘻嘻]

而对于后面的提权，只是因为我最初接触这个行业的时候是作为一个CTF选手去解决reverse方向的题目所以做了一道反编译题目。